Verilog Review

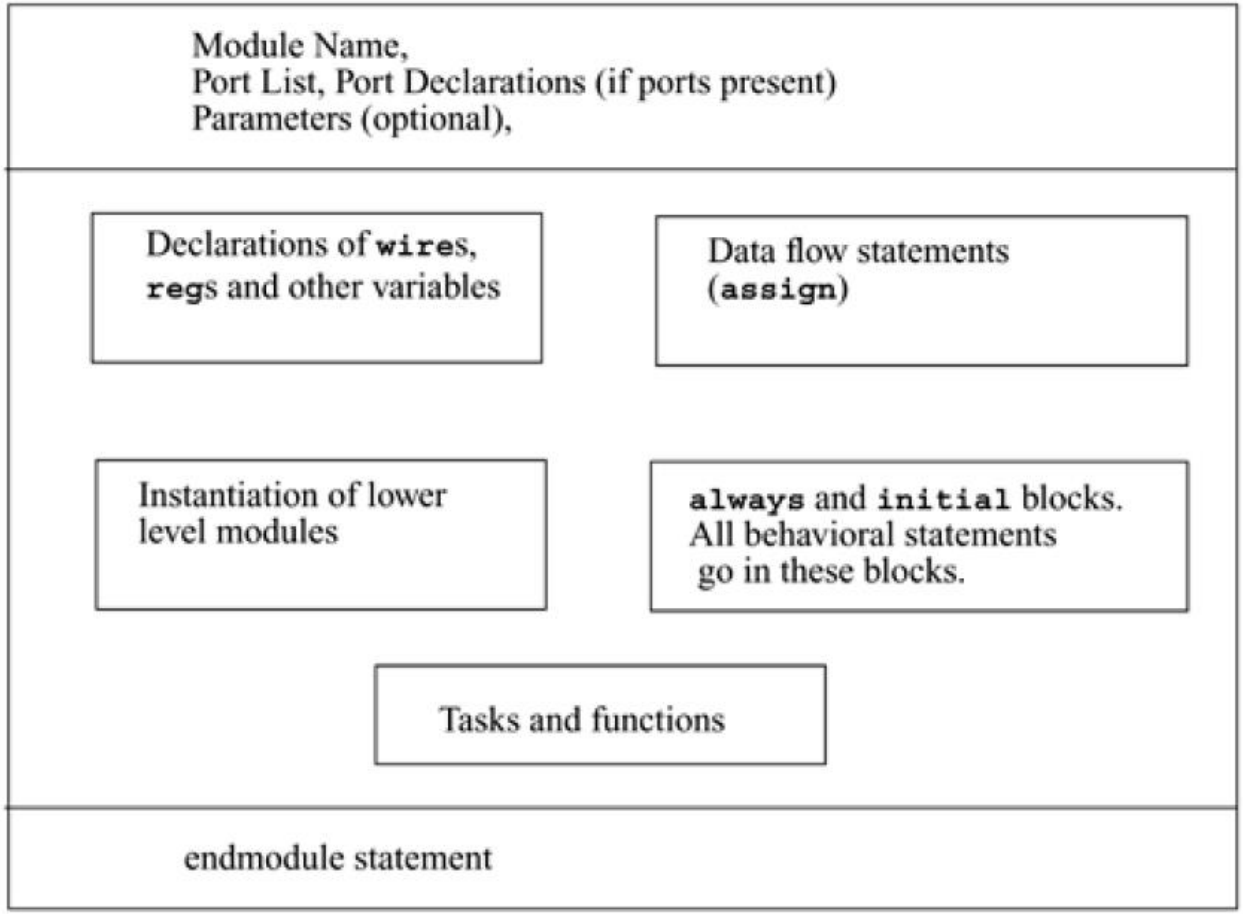
Outlines

- overview
- signed vs. unsigned
- dataflow, structural, behavioral modeling
- RCA
- pipelined MAD
- LOD
- testbench

Overview

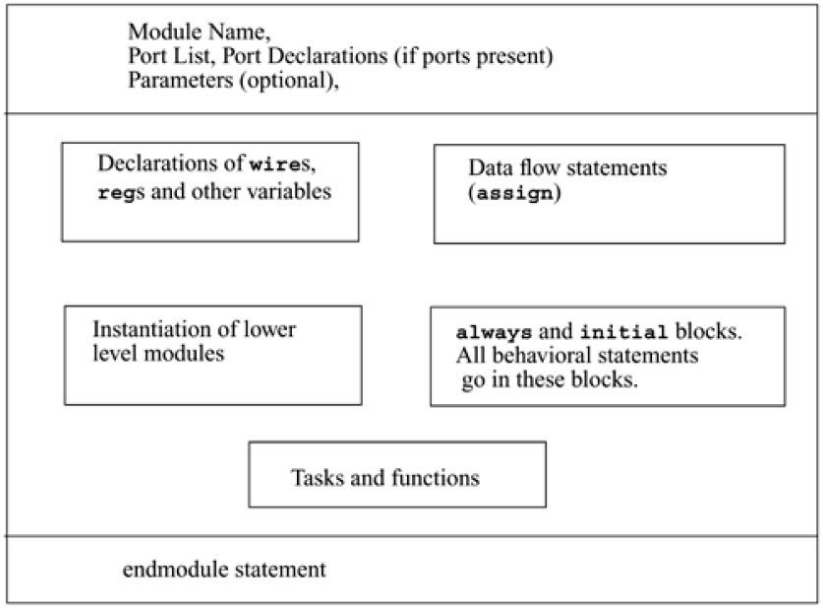
Components of a Verilog Module

- declaration
  - inputs, outputs
  - parameters
  - data types
- structure model
- module instantiation
- dataflow model
  - continuous assignment
- behavior model
  - procedural assignment



```
module M (P1, P2, P3, P4);
input P1, P2;
output [7:0] P3;
inout P4;

reg [7:0] R1, M1[0:1023];
wire W1, W2, W3, W4;
wire [3:0] W5;
parameter C1=const;

// non-synthesizable coding
// used in testbench
initial
begin : blockname
  // statements
end

// behavioral modeling
always
begin
  // statements
end
```

```
// dataflow: continuous assignment
assign W1 = expr ;  // use operators

// module instances: structural
modeling
COMP U1 (.PP2(W2), .PP1(W1);
COMP U2 (W3, W4);

task T1;
input A1, A2;
output A3, A4;
begin
  // statements
end
endtask

function [7:0] F1;
input A1;
begin
  // statements
end
endfunction
```



Combinational VS. Sequential Logic

- Combinational Logic
  - Logic circuits with pure feed-forward datapath
  - No flip flops or latches
  - e.g.

```
module adder (input wire [7:0], a, b, output  wire [7:0] c);
assign c = a + b;   // dataflow modeling
endmodule
```

- Sequential Logic
  - Logic circuits with latches or flip-flops
  - e.g.

```
module lateched (input [7:0], a, b, input  clk, output reg [7:0] c);
always @ (poedege clk)    // behavioral modeling
    c = a + b;  // result c stored in a positive-edge-riggered register
endmodule
```

synthesizable vs. non-synthesizable

- synthesizable codes
  - used to describe hardware design
  - continuous assignments (**assign** …) in dataflow model
    - ✓ with various operators
    - ✓ left-hand-side is **wire** type
  - procedure assignments (**always @** …) in behavioral model
    - ✓ **if … else… , case … endcase , for … , while … ,**
      - left-hand-side is **reg** type
  - module (built-in gate) instantiation in structural model
- non-synthesizable codes
  - used in testbench for verification
  - **initial …** , timing control (**#** …) , file access, …

module definition

- input, output ports
- Register Transfer Level (RTL) modeling
  - combination of dataflow, structural, behavioral statements

```
module MyCPU(R_Wb, Data, Address, Clock, Reset);
  input  Reset, Clock;
  input  [31:0] Address;
  inout  [31:0] Data;
  output R_Wb;

        .
        .
        .

        .
        .
        .
endmodule
```

I/O port declarations

Resource / variable declarations

Structural / behavioral modeling

numbers, data types, operators

- numbers
  - sized numbers, **<size>’<base format> <number>**
    - ✓ e.g., 4’b1111, 12’habc, 16’d255
  - unsized numbers (decimal by default)
    - ✓ e.g., wire [7:0] b = a + 3; // decimal value 3 is extended to 8-bit
- data types
  - wire (default), reg,
  - bit-vector, e.g., wire]7:0] a, b; reg]7:0] c;
  - array, e.g., wire [7:0] image [1:480][1:360]; // VGA resolution
- operators
  - arithmetic, e.g., c=a+b; d=a-b; e=a*b;
  - bit-wise, e.g., wire [7:0] c=a&b, // a, b are 8-bit numbers
  - shift, e.g., c = a>>sc, d=a>>>sc, e=a<<sc;
  - …

Structural, Dataflow, Behavioral Modeling

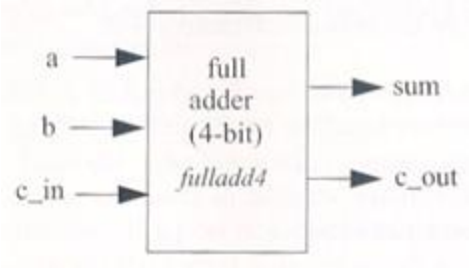
- structural modeling
  - instantiation of other modules
  - e.g. FA instance_name (sum, cout, a, b, cin);
- dataflow modeling
  - continuous assignment **assign**
  - e.g., **assign** {cout, sum} = a + b + cin;
- behavioral modeling
  - procedural assignments inside always @(…) block
  - e.g. 1-bit full adder

```
reg sum, cout;

always @ (a, b, cin)
   {cout, sum} = a + b + cin;
```

dataflow model (assign)

- continuous assignment **assign**
  - describe combinational logic
  - left hand side must be **wire** data type (default type)
- operators
  - unary operators, e.g., assign b = ~a; // one operand
  - binary operators, e.g., assign c = a+b; // two operands
  - unary conditional operator, e.g., assign d = c? a:b;
  - concatenation operator, e.g., assign c = {2'b10, 2{a}, b}
    - ✓ equivalent to assign c = {2'b10, a, a, b} // c=11_a_a_b
- implicit assignment
  - wire [7:0] c = a+b;
    - ✓ equivalent to two statements: wire [7:0] c; assign c = a+b;

structural model



- port and parameter declaration

```
// IEEE 1364-1995 (old)
module fulladd4(sum, c_out, a, b, c_in);
parameter width = 4;
output [width-1:0] sum;
output c_out;
input [width-1:0] a, b;
input c_in;
reg [width-1:0] sum;
reg c_out;
…
endmodule
```

```
// ANSI C style (IEEE 1364-2001)
module
    # (parameter width =4)
    fulladd4 (
    output reg [width-1:0] sum,
    output reg c_out,
    input [width-1:0] a, b,
    input c_in
);
…
endmodule
```

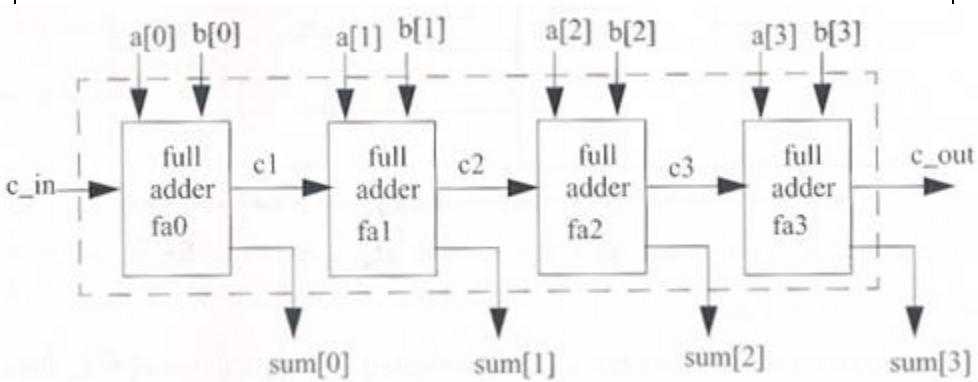
- port instantiation
  - connection by order
    - ✓ e.g., **fulladd4 fa_ordered (ext_s, ext_co, ext_a, ext_b, ext_ci);**
  - connection by name
    - ✓ e.g., **fulladd4 fa_byname (.c_out(ext_co), .sum(ext_s), .b(ext_b), .c_in(ext_ci), .a(ext_a));**

behavioral model (always, initial)

- inside **always** or **initial** block
  - initial blocks are non-synthesizable, only for testbench
  - could describe sequential logic or combinational logic
- behavioral statements
  - left-hand-side must be **reg** data type
  - procedural assignment **<=** with event/timing control **@**
    - ✓ e.g., **reg [7:0] a, b, c; always @ (posedge clk) c <= a+b;**
  - loop statements
    - ✓ for, while, …
  - conditional statements or multi-way branch
    - ✓ **if … else** …
    - ✓ **case … endcase**

4-bit adder

```
// structural level modeling
module fulladd4 (sum, c_out, a, b, c_in);

output [3:0] sum;
output c_out;
input [3:0] a, b;
input c_in

wire c1, c2,c3;
// construct 4-bit adder fullladd4 from
// previously defined module fulladd

        fulladd   fa0 (sum[0], c1, a[0], b[0], c_in);
        fulladd   fa1 (sum[1], c2, a[1], b[1], c1);
        fulladd   fa2 (sum[2], c3, a[2], b[2], c2);
        fulladd   fa3 (sum[3], c_out, a[3], b[3], c3);

endmoudle
```




```
// dataflow level modeling
// using continuous assignment
module FA_dataflow
    (output [3:0] sum,
     output  c_out,
     input [3:0] a, b,
     input c_in);

     assign {c_out, sum} = a+b+c_in;
// synthesized gate netlist depends on
// area/delay constraints from users

endmodule
```

```
// behavioral level modeling
module FA_behavior
    (output reg [3:0] sum,
     output reg c_out,
     input [3:0] a, b,
     input c_in);

     always @(a, b, c_in)
            {c_out, sum} = a+b+c_in;

endmodule
```

signed vs. unsigned

- unsigned (default Verilog data type)

```
wire [3:0] x;  wire [7:0] y;
wire [7:0] a, b, s;  wire [8:0] s1;

assign y=x;  // zero padded =>  assign y = {4'b0000, x};
assign  s=a+b; // no overflow detection => 1111_1111+0000_0001=0000_0000
assign s1=a+b; // s1 = {1'b0, a} + {1b'0, b}; implicitly zero-extension
```

- signed (new data type added in 2001)

```
wire signed [3:0] x;     wire signed [7:0] y;
wire sigmed [7:0] a, b, s;   wire signed [8:0] s1;

assign y=x;  // sign-bit padded =>  assign y = {4{x]3]}, x};
assign  s=a+b; // no overflow detection => 1111_1111+1000_0001=0111_1111
assign s1=a+b; // s1 = {a[7], a} + {b[7], b}; implicitly sign-extension
```

unsigned multi-operand adder

- unsigned numbers are non-negative integers
- **zero**-extension inputs to enough bit-width for output
  - when adding p numbers, add $r=\log_2 p$ zero bits

```
module #(parameter m=4, n=4, r=2) add4
    (input [m+n-1:0] a, b, c, d, output [m+n+r-1:0] s);
wire [m+n+r-1:0] A={2'b0, a};  // add r zero bits for Q(m.n), i.e., 00_a
wire [m+n+r-1:0] B={2'b0, b};   // 00_b
wire [m+n+r-1:0] C={2{2'b0, c};  // 00_c
wire [m+n+r-1:0] D={2'b0, d};
assign s = A+B+C+D;   // RHS and LHS have same bit-width
endmodule
// module instance: add4 (16, 16, 2) 32b_adder (x,y, z, w, out);
```

```
module #(parameter m=4, n=4, r=2) add4
    (input wire [nm+n-1:0] a, b, c, d, output wire [m+n+r-1:0] s);
assign s = a+b+c+d;  // automatically zero-extend inputs a, b, c, d
endmodule
```

signed multi-operand adder

- signed numbers could be positive or negative integers
- **sign**-extension inputs to enough bit-width for output
  - when adding p numbers, sign-extend $r=\log_2 p$ bits

```
module #(parameter m=4, n=4, r=2) add4
    (input [m+n-1:0] a, b, c, d, output [m+n+r-1:0] s);
wire [m+n+r-1:0] A={2{a[m+n-1]}, a};  // 2 sign-extend r bits for Q(m.n)
wire m+n+r-1 [:0] B={2{b[m+n-1]}, b};
wire [m+n+r-1:0] C={2{c[m+n-1]}, c};
wire m+n+r-10] D={2{d[m+n-1]}, d};
assign s = A+B+C+D;
endmodule
// module instance: add4 (16, 16, 2) 32b_adder (x,y, z, w, out);
```

```
module #(parameter m=4, n=4, r=2) add4
    (input signed [m+n-1:0] a, b, c, d, output signed [m+n+r-1:0] s);
assign s = a+b+c+d;  // automatically sign-extend inputs a, b, c, d
endmodule
```

n-bit Ripple Carry Adder (RCA)

- generate loop in structural modeling

```
generate
   for (i=0; i<=n-1; i=i+1)
      begin: FA_loop  // block named “FA_loop”
         fulladd  fa (sum[i], c[i+1], a[i], b[i], c[i]);
         // fulladd FA_loop[0].fa ( sum[0], c[1], a[0], b[0], c[0] );
         // fulladd FA_loop[1].fa ( sum[1], c[2], a[1], b[1], c[1] );
         // fulladd FA_loop[2].fa ( sum[2], c[3], a[2], b[2], c[2] );
         // fulladd FA_loop[3].fa ( sum[3], c[4], a[3], b[3], c[3] );
      end
endgenerate
// fulladd FA_loop[0:3] (sum[0:3], c[1:4], a[0:3], b[0:3], c[0:3]);
```

- for loop in behavioral modeling

```
for (i=0; i<=n-1; i=i+1 {c[i+1],
      {c[i+1], sum[i]} = a[i]+b[i]+c[i];
```

Pipelined Multiply-Add (MAD)

- two pipelined stages with two always blocks

```
module MAD (input [n-1] a, b, input [2n-1:0] c,  output reg [2*n-1:0] mad);
parameter n=16;
wire clk;
reg [2*n-1:0] p, c_tmp;

// 1st  pipeline stage that performs multiplication
always @ (posedge clk) begin
 p <= a*b;          // register to store the product a*b
c_tmp <= c;      // register to store c, to be used in 2nd stage
end

// 2nd pipeline stage that performs addition
always @ (posedge clk) begin
   mad <= p + c_tmp;
end
endmodule
```

Leading One Detector (LOD)

- normalization in floating-point arithmetic
- finding maximum in a block of numbers
  - e.g., using for loop to model LOD

```
…
wire [n-1:0] d;
always@(*) begin   // count number of leading zeros for d[n-1:0]
  continue = `TRUE;
  for (i=n-1; continue && (i>=0); i=i-1) begin
    zero_cnt = din[i] ? (n-1) - i : 0;
    continue = din[i] ? `FALSE  : `TRUE;
  end
…
```

another example: DP4

- 4-D dot-product: a[0]*b[0]+ a[1]*b[1]+ a[2]*b[2]+ a[3]*b[3]

```
module DP4 (input[16*4-1:0]in_a, in_b, output [31:0]out);
wire [15:0] a[3:0],  b[3:0];

// convert two long bit-vectors in_a, in_b into two arrays a, b
assign {a[3],a[2],a[1],a[0]}=in_a;
assign {b[3],b[2],b[1],b[0]}=in_b;

integer i;
reg [31:0] tmp[0:3];
always @ (*) begin
   tmp[0] = a[0]*b[0];
   for (i=1; i<=3; i=i+1) tmp[i] = a[i]*b[i] + tmp[i-1];
end

assign out = tmp[3];

endmodule
```

another example: DPn

- n-D dot-product

```
module #parameter n=4 DPn (input[16*n-1:0]in_a, in_b, output [31:0]out);
reg [15:0] a[n-1:0];
reg [15:0] b[n-1:0];
integer i;
always@ (*) begin   // convert two long bit-vectors into two arrays
            for (i=0; i<n; i=i+1) begin
                        a[i]= in_a[16*i +:16];
                        b[i]= in_b[16*i +:16];
            end
end

reg [31:0] tmp[0:n];
always @ (*) begin
   tmp[0] = 0;
   for (i=0; i<=n-1; i=i+1) tmp[i+1] = a[i]*b[i] + tmp[i];
end
assign out = tmp[n];
endmodule
```

16-bit LOD using casex (~45 gates)

```
module LZD_case (nput [15:0]din,  output reg [3:0]zero_cnt);
always@(*) begin
  casex(din)
  16'b1xxx_xxxx_xxxx_xxxx : zero_cnt = 4'd0;
  16'b01xx_xxxx_xxxx_xxxx : zero_cnt = 4'd1;
  16'b001x_xxxx_xxxx_xxxx : zero_cnt = 4'd2;
  16'b0001_xxxx_xxxx_xxxx : zero_cnt = 4'd3;
  16'b0000_1xxx_xxxx_xxxx : zero_cnt = 4'd4;
  16'b0000_01xx_xxxx_xxxx : zero_cnt = 4'd5;
  16'b0000_001x_xxxx_xxxx : zero_cnt = 4'd6;
  16'b0000_0001_xxxx_xxxx : zero_cnt = 4'd7;
  16'b0000_0000_1xxx_xxxx : zero_cnt = 4'd8;
  16'b0000_0000_01xx_xxxx : zero_cnt = 4'd9;
  16'b0000_0000_001x_xxxx : zero_cnt = 4'd10;
  16'b0000_0000_0001_xxxx : zero_cnt = 4'd11;
  16'b0000_0000_0000_1xxx : zero_cnt = 4'd12;
  16'b0000_0000_0000_01xx : zero_cnt = 4'd13;
  16'b0000_0000_0000_001x : zero_cnt = 4'd14;
  16'b0000_0000_0000_0001 : zero_cnt = 4'd15;
  default : zero_cnt = 4'd0;
  endcase
end
endmodule
```

Delay=0.60ns

Area=126 um x um (um^2)
~=(126/2.82) NAND2 gates
~= 45 gate equivalent

Hierarchical cell	Global cell area: Absolute Total	Global cell area: Percent Total	Local cell area: Combi-national	Local cell area: Noncombi-national	Local cell area: Black-boxes	Design
LZD top	339.3936	100.0	0.0000	0.0000	0.0000	LZD top
u LZD case	126.3024	37.2	126.3024	0.0000	0.0000	LZD case
u LZD for	106.5456	31.4	106.5456	0.0000	0.0000	LZD for
u LZD while	106.5456	31.4	106.5456	0.0000	0.0000	LZD while

16-bit LOD using for loop (~38 gates)

Hierarchical cell	Global cell area: Absolute Total	Global cell area: Percent Total	Local cell area: Combi-national	Local cell area: Noncombi-national	Local cell area: Black-boxes	Design
LZD top	339.3936	100.0	0.0000	0.0000	0.0000	LZD top
u LZD case	126.3024	37.2	126.3024	0.0000	0.0000	LZD case
u LZD for	106.5456	31.4	106.5456	0.0000	0.0000	LZD for
u LZD while	106.5456	31.4	106.5456	0.0000	0.0000	LZD while
Total			339.3936	0.0000	0.0000	

```
`define TRUE 1'b1
`define FALSE 1'b0

module LZD_for (
input [15:0]din, output reg [3:0]zero_cnt); input [15:0] din, output reg [3:0] zero_cnt);
reg continue;
integer i;

// zero_cnt is the number of leading zeros
// continue is false, if leading one is found
always@(*) begin
  continue = `TRUE;
  for (i=15; continue && (i>=0); i=i-1) begin
    zero_cnt = din[i] ? 4'd15 - i : 4'd0;
    continue = din[i] ? `FALSE  : `TRUE;
  end
end

endmodule
```

Delay=0.33ns

Area=106 um x um (um^2)
~=(106/2.82) NAND2 gates
~=38 gate equivalent

16-bit LOD using while loop (~38 gates)

```
`define TRUE 1'b1
`define FALSE 1'b0

module LZD_while ( input [15:0]din,    output reg[3:0]zero_cnt);
reg continue;
integer i;

// zero_cnt is the number of leading zeros
// continue is false, if leading one is found
always@(*) begin
  i = 15;
  continue = `TRUE;
  while ((i>=0) && continue) begin
    zero_cnt = din[i] ? 4'd15 - i : 4'd0;
    continue = din[i] ? `FALSE : `TRUE;
    i=i-1;  // problematic ?
  end
end

endmodule
```

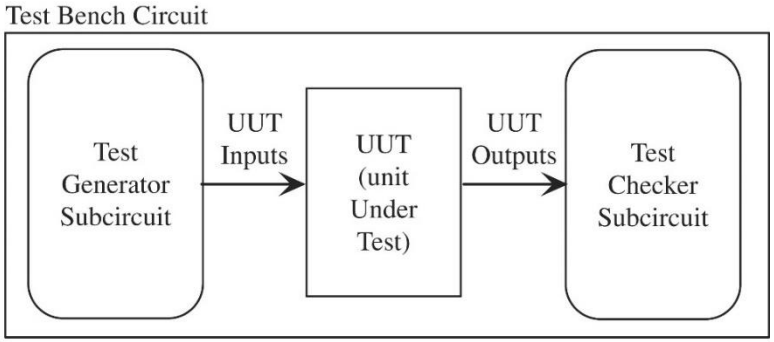
Hierarchical cell	Global cell area: Absolute Total	Global cell area: Percent Total	Local cell area: Combi-national	Local cell area: Noncombi-national	Local cell area: Black-boxes	Design
LZD top	339.3936	100.0	0.0000	0.0000	0.0000	LZD top
u LZD case	126.3024	37.2	126.3024	0.0000	0.0000	LZD case
u LZD for	106.5456	31.4	106.5456	0.0000	0.0000	LZD for
u LZD while	106.5456	31.4	106.5456	0.0000	0.0000	LZD while
Total			339.3936	0.0000	0.0000	

Delay=0.33ns

Area=106 um x um (um^2)
~=(106/2.82) NAND2 gates
~=38 gate equivalent

Testbench

- instantiate hardware design block
  - map input/output pins
- specify inputs
  - could directly in testbench, or
  - read from test vector files, or
  - randomly generate inputs
- apply test input test patterns and get output results
  - connect the specified inputs to the instantiated hardware
  - obtain the simulation outputs
- compare simulation outputs with expected results
  - expected could be from test vector file, or
  - generated using non-synthesizable codes in testbench



Test Patterns

- Two ways to generate inputs, expected results
- access files
  - use file access system tasks
    - ✓ e.g., $readmemh, $fscanf to read data
    - ✓ e.g., $fwrite to write results
  - might need other programs (e.g., C, Python) to generate input files
  - might need other programs to verify simulation results in the output file
  - the previous example read test patterns and expected results from a file, and make comparison in the testbench
- inside testbench module
  - randomly generate input test patterns
    - ✓ e.g., $random
  - could use non-synthesizable Verilog codes to generate expected results (e.g., real x, y, z ‘ assign z=x+y; // floating-pt. add)

testbench.v (1/2)

```
`timescale 1ns/10ps
`define CYCLE 50.0
`define DATA_NUM 100
`define FILE_A "./a.txt"
`define FILE_B "./b.txt"
`define FILE_D "./d.txt"
module testbench();
   integer file_a, file_b, file_d;

   reg CLK = 0;
   reg RST = 0;
   reg [31:0] data_a [0:`DATA_NUM - 1];
   reg [31:0] data_b [0:`DATA_NUM - 1];
   reg [31:0] input_a;
   reg [31:0] input_b;
   wire [31:0] outcome;
   reg [31:0] answer;

   FXP_adder test_module( .a(input_a), .b(input_b), .d(outcome));

   //Post_sim 使用，在nc_post_syn.f 中define SDF_FILE與SDF_FILE路徑
   `ifdef SDF_FILE
      initial $sdf_annotate(`SDF_FILE, test_module);
   `endif
   // 設定clock訊號
   always begin #(`CYCLE/2) CLK = ~CLK; end

   integer i, flag=0, error=0, garbage;

   //將file_a與file_b中資料存入data_a與data_b陣列中
   initial begin
      file_a = $fopen(   `FILE_A , "r");    // file_a=$fopen("a.txt", "r");
      file_b = $fopen(   `FILE_B , "r");    // file_b=$fopen("b.txt", "r");
      file_d = $fopen(   `FILE_D , "w");    // file_d=$fopen("d.txt", "w");
      for (i = 0; i < `DATA_NUM; i=i+1)
      begin
         garbage = $fscanf(file_a, "%X", data_a[i]);
         garbage = $fscanf(file_b, "%X", data_b[i]);
      end
   end
```

```
module FXP_adder(
input signed [31:0] a, b,
   output signed [31:0]
                  d );
   assign d = a + b;
endmodule
```

```
`
   initial begin
      //File approach
      $display("-----------------------------------\n");
      $display("-        FXP_ADDER_USE_FILE        -\n");
      $display("-----------------------------------\n");

      CLK = 0;
      RST = 1;
      #(`CYCLE*2);
      RST = 0;
      for(i=0; i<`DATA_NUM; i=i+1)
      begin
         input_a = data_a[i];
         input_b = data_b[i];
         answer = $signed(input_a) + $signed(input_b);
         #(`CYCLE);
         $fwrite(file_d, "%X\n", outcome);
         if(answer !== outcome)
         begin
            error = error+1;
            if(1 || flag==0)
            begin
               $display("-----------------------------------\n");
               $display("Output error at #%d\n", i+1);
               $display("The input A is    : %X\n", input_a);
               $display("The input B is    : %X\n", input_b);
               $display("The answer is     : %X\n", answer);
               $display("Your module output: %X\n", outcome);
               $display("-----------------------------------\n");
               flag = 1;
            end //if flag
         end //if
      end //for
      if(flag==1)//if wrong
      begin
         $display("Total %4d error in %4d testdata.\n", error, i);
         $display("-----------------------------------\n");
      end//if
      else
```

testbench.v (2/2)

```
`    begin//if right
        $display("-----------------------------------\n");
        $display("All testdata correct!\n");
        $display("-----------------------------------\n");
    end//else

    //random approach
    $display("-----------------------------------\n");
    $display("-        FXP_ADDER_USE_RANDOM        -\n");
    $display("-----------------------------------\n");

    flag=0;
    error=0;
    CLK = 0;
    RST = 1;
    #(`CYCLE*2);
    RST = 0;
    for(i=0; i<`DATA_NUM; i=i+1)
    begin
        input_a = $random % 2**31 ; //產生介於 -(2^31)-1到(2^31)-1的數
        input_b = $random % 2**31 ;
        answer = $signed(input_a) + $signed(input_b);
        #(`CYCLE);
        if(answer !== outcome)
        begin
            error = error+1;
            if(1 || flag==0)
            begin
                $display("-----------------------------------\n");
                $display("Output error at #%d\n", i+1);
                $display("The input A is    : %X\n", input_a);
                $display("The input B is    : %X\n", input_b);
                $display("The answer is     : %X\n", answer);
                $display("Your module output: %X\n", outcome);
                $display("-----------------------------------\n");
                flag = 1;
            end //if flag
        end //if
    end //for
```

```
    if(flag==1)//if wrong
        begin
            $display("Total %4d error in %4d testdata.\n", error, i);
            $display("-----------------------------------\n");
        end//if
        else
        begin//if right
            $display("-----------------------------------\n");
            $display("All testdata correct!\n");
            $display("-----------------------------------\n");
        end//else

    $fclose(file_a);
    $fclose(file_b);
    $fclose(file_d);
    $finish;
    end

endmodule
```

Input a.txt

```
558b4567
51bc9869
5a30dc51
3568944a
318e1f29
3e9b58ba
373141f2
5ce2a9e3
485f007c
29200854
2116231b
2890cde7
```

Input b.txt

```
397b23c6
52b34873
2cc95cff
50d558ec
43687ccd
47fed7ab
40b71efb
5a45e146
4dd062c2
46b127f8
2f96e9e8
536f438d
```

output d.txt

```
08f06692d
0a46fe0dc
086fa3950
0863ded36
074f69bf6
0869a3065
077e860ed
0b7288b29
0962f633e
06fd1304c
050ad0d03
07c001174
```

```
… // instantiate hardware
FXP_adder test_module ( .a(input_a), .b(input_b), .d(outcome));
… // read test patterns from files
file_a = $fopen( “a.txt” , "r");
file_b = $fopen( “b.txt”  , "r");
file_d = $fopen(“c.txt” , "w");
for (i = 0; i <100; i=i+1) begin …
   garbage = $fscanf(file_a, “%X”, data_a[i]);   // hexa-decimal format
   garbage = $fscanf(file_b, “%X”, data_b[i]); … end
… // compute expected answer, get hardware outputs and write to a file
for (i=0; i< 100; i=i+1) begin …
   input_a = data_a[i];   // apply test patterns to hardware inputs
   input_b = data_b[i];
   answer = $signed(input_a) + $signed(input_b); // expected results
   if (outcome != answer) ….    // error occurs
   $fwrite(file_d, "%X \n", outcome);  … end
… // an alternative of generating input test patterns randomly
 for (i=0; i<100; i=i+1) begin …
   input_a = $random % (2**31);   // between -2^31 + 1 and 2^31 – 1
   input_b = $random % (2**31);
   answer = $signed(input_a) + $signed(input_b); … end
…
```

System Task $

- read from input
  - **$readmemb, $readmemh**
  - **$fscanf, $fget. $fread**
- write to output
  - $fopen, $fclose
  - **$display, $strobe, $monitor, $write**
  - **$fdisplay, $fstrobe, $fmonitor, $fwrite**
- random generator
  - **$random**
- suspend (**$stop**) or finish (**$finish**) simulation
- conversion functions
  - **$signed, $unsigned** (synthesizable)

Random Number Generation ($random)

- *Seed* can be either **reg, integer,** or **time** variable
- **$random** returns a 32-bit *signed* integer

```
module test;
integer r_seed;
reg [31:0] addr;  // input to ROM
wire [31:0] data; // output from ROM
reg [23:0] rand1, rand2;
…
ROM rom1 (data, addr);
…
initial   r_seed = 2;  // arbitrarily define seed as 2, better use different seed values
always @ (posedge clock)
      addr  = $random (r_seed); // generate random numbers of signed integer
      rand1 = $random % 60; // generate random number between -59 and 59
      rand2 = {$random} % 60; // random number between 0 and 59
…
// check output of ROM against expected results
…
endmodule
```

Compiler Directives ` (back quote)

- **`define**
  – define text macro for later text macro substitution
  `define WORD_SIZE 32
  // used as **`WORD_SIZE** in the code such as
  // **wire** [**`WORD_SIZE**-1:0] a, b, c;
- **`include**
  – include another Verilog file, e.g.,
  `include header.v //include the file header.v
- **`ifdef**
  – conditional compilation
  `ifdef TEST **module** test;
  // compile module test only if text macro TEST
  // is defined using **`define** TEST
- **`timescale**
  `timescale 100ns/1ns